EFREI Paris

ST2OOS, academic year 2026/2027

QuantBacktest

Financial Strategy Backtesting Platform

Project report

Alexandre Tung	Data, reports, deployment
Victor Haegeman	Strategies, no-code rules, Monte Carlo
Eloi Dumas	Backtesting engine, metrics, optimisation
Alexis Launay-Schapman	Web application

14 November 2026

Contents

1 Introduction **3**

2 Architecture **3**

2.1 Layers and contract . . . 3

2.2 Request flow . . . 4

2.3 Working in parallel . . . 4

2.4 Conventions . . . 5

3 Data layer **5**

3.1 What the free plan gives . . . 5

3.2 Cache, seed files and loading . . . 5

3.3 Talking to Alpha Vantage safely . . . 6

3.4 Parsing, quota and concurrency . . . 7

3.5 Data quality . . . 7

4 Strategies **8**

4.1 What a strategy is allowed to do . . . 8

4.2 Indicators . . . 8

4.3 The implemented strategies . . . 9

4.4 Results on SPY . . . 12

4.5 Custom strategies . . . 12

5 Backtesting engine **13**

5.1 Execution model . . . 13

5.2 Advanced orders . . . 14

5.3 Metrics . . . 15

5.4 Optimisation heatmap . . . 15

6 Robustness analysis **16**

6.1 Train/test validation . . . 16

6.2 Monte Carlo simulation . . . 17

7 Web application **18**

8 Reports and deployment **20**

8.1 PDF report . . . 20

8.2 Docker . . . 21

9 Quality and teamwork **22**

9.1 Tests 22

9.2 Workflow and continuous integration 22

9.3 What we learned 23

10 Limitations and future work **23**

1 Introduction

A backtest answers one simple question: if I had followed this trading rule in the past, what would have happened to my money? The goal of this project was to build a backtesting website in C# and .NET. The subject asked us to fetch historical prices from the Alpha Vantage API, to store them locally in order to limit API calls, to implement at least six trading strategies, to simulate them on daily and weekly data, to compute performance metrics, to show the results in a web interface and to generate reports with graphs and trade logs.

Requirement of the subject	**In QuantBacktest**
Fetch data from the free Alpha Vantage API	Typed HTTP client, daily and weekly series (Section 3)
Store data locally	SQLite cache and seed files
At least 6 strategies	10 strategies and a buy & hold baseline (Section 4)
Custom strategies from templates	A C# template and a no-code rule builder
Simulation engine, daily and weekly	Next-open execution model (Section 5)
Return, volatility, drawdown, Sharpe	13 metrics, for the strategy and for buy & hold
Web interface with charts and tables	ASP.NET Core MVC application (Section 7)
Reports with graphs and trade logs	PDF report and CSV export (Section 8)

Table 1: Requirements of the subject and where they are met.

Beyond the subject, we added features whose purpose is mostly to help the user trust a result, or to distrust it: stop-loss, take-profit, trailing stop and slippage; an optimisation heatmap with an overfitting warning; train/test validation; a comparator of strategies; advanced metrics (Sortino, Calmar, profit factor, exposure); indicators drawn on the charts and an explanatory card for each strategy; a Monte Carlo simulation on the trades; a data quality panel and an API quota counter; a permanent link for each result; and a Docker image.

The interface is presented in the demonstration video, so this report focuses on how the platform works, on the choices we made, and on what its results mean.

Member	**Responsibility**
Alexandre Tung	Alpha Vantage client, SQLite cache, seed files, data quality, API quota, storage of no-code strategies, PDF and CSV reports, Docker
Victor Haegeman	Technical indicators, strategies, C# template, no-code rule engine, Monte Carlo
Eloi Dumas	Backtesting engine, advanced orders, metrics, optimisation heatmap
Alexis Launay-Schapman	Web application, comparator, train/test, permanent links

Table 2: Split of the work.

2 Architecture

2.1 Layers and contract

Four people had to work at the same time without blocking each other, and most of the architecture comes from this constraint. The solution contains one shared project, `Core`, and one project per responsibility (Figure 1).

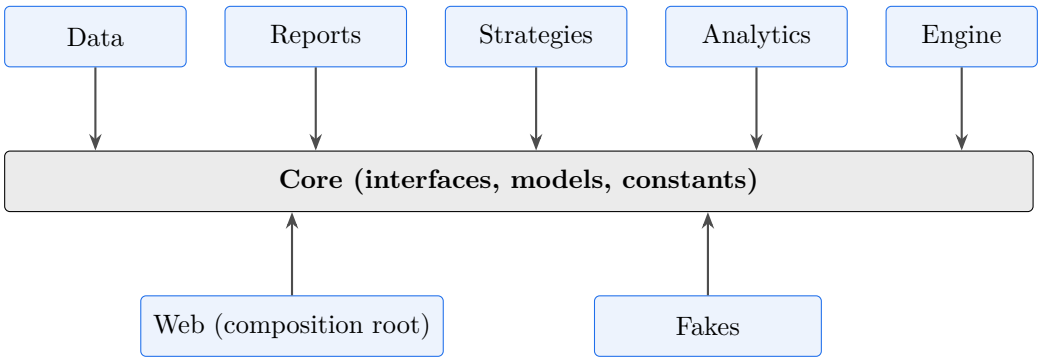


Figure 1: Project dependencies. Every project depends on `Core` only; `Web` also references the others, only to plug them in.

`Core` contains only interfaces (`IMarketDataService`, `IStrategy`, `IBacktestEngine`...), data models (`PriceSeries`, `BacktestResult`...) and constants. A method signature cannot say everything: it cannot say, for example, at which price an order is filled. So we also wrote a contract with numbered rules (X1–X15 for the execution of orders, M1–M14 for the metrics, D1–D8 for the data, and so on), which the code and the tests quote. Once validated, it was frozen: changing it required the agreement of all four members. In the end nobody had to change it.

2.2 Request flow

A project never references the project of another part, only `Core`. The parts communicate through interfaces injected by dependency injection, and only `Web`, the composition root, knows every project. Figure 2 shows one backtest.

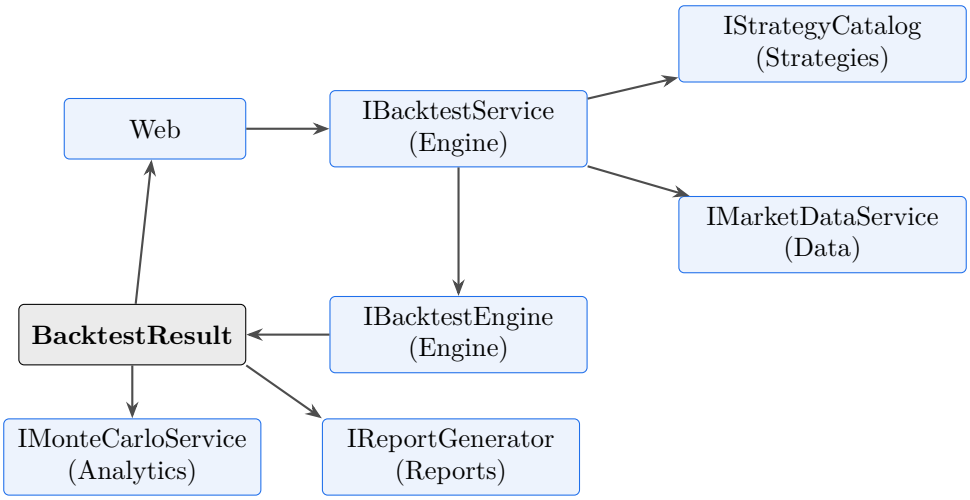


Figure 2: Flow of one backtest. Each box is an interface of `Core` and the project implementing it.

2.3 Working in parallel

The `Fakes` project contains a plausible fake of every interface and a deterministic price generator. Because of it, the web pages could be built before the real engine existed. One configuration key per part chooses between the real service and the fake, so each real part was plugged in as soon as it was ready. The fakes are only a development tool: the contract defines the correct behaviour.

2.4 Conventions

A few conventions, applied everywhere, removed whole families of bugs:

- `decimal` for prices, quantities and amounts, and `double` for returns and ratios;
- fractions in the code (0.12 means 12 %), percentages only on screen;
- `CultureInfo.InvariantCulture` whenever a number or a date is parsed or written outside the display;
- `DateOnly` for market dates, and an injected `TimeProvider` instead of `DateTime.Now`;
- an undefined ratio is `null` and displayed as "n/a", never `NaN` and never replaced by 0.

3 Data layer

3.1 What the free plan gives

Before designing anything, we tested the endpoints with a free key (Table 3). The result changed our plans: the full daily history and the adjusted daily series are premium features.

Endpoint	Result	Period	Bars
`TIME_SERIES_DAILY`, compact	Accepted	2026-05-22 to 2026-10-14	100
`TIME_SERIES_DAILY`, full	Refused (premium)		–
`TIME_SERIES_DAILY_ADJUSTED`	Refused (premium)		–
`TIME_SERIES_WEEKLY`	Accepted	1999-11-12 to 2026-10-14	1406
`TIME_SERIES_WEEKLY_ADJUSTED`	Accepted	1999-11-12 to 2026-10-14	1406

Table 3: Endpoints tested on the free plan (MSFT, 15 October 2026).

We therefore use the compact daily endpoint and the adjusted weekly endpoint. Weekly data gives 27 years of prices adjusted for splits and dividends, so long tests use it. The adjustment applies the factor *adjusted close / close* to the four prices and leaves the volume unchanged. A series is marked as adjusted only if all of its bars are (rule D6). We did not add a third-party source for long daily histories, to avoid any licensing doubt.

3.2 Cache, seed files and loading

Prices live in a local SQLite database accessed with EF Core, with four tables: `PriceBars`, `SeriesMetadata`, `ApiUsage` and `CustomStrategies` (Figure 3). Loading a series follows these rules (D2, D3):

1. **Without refresh, cache not empty:** the cache is returned and no API call is made, so a backtest gives the same result every time.
2. **Without refresh, cache empty:** the seed file of the series is imported if it exists; otherwise the API is called.
3. **With refresh:** the API is called and the bars are merged by date. New dates are inserted, existing dates are replaced, and no bar is ever deleted. If the call fails, the cache is left untouched.

The seed files are snapshots of the free endpoints for AAPL, MSFT, SPY and JPM, in daily and weekly. They let the application work without an API key. Each series records its origin: `Seed`, `Api` or `Mixed`. Figure 4 shows a real refresh.

```
PS C:\Users\tunga> python "C:\Users\tunga\AppData\Local\Temp\claude\e6a9a34502162c4d\74497b4f-22cd-410b-afa9-49024e1acd51\scra
tchpad\show_tables.py"
Database: C:/Users/tunga/Downloads/QuantBacktest-DataDemo/App_Data/demo.db

Tables and row counts
  ApiUsage               1 rows
  CustomStrategies       0 rows
  PriceBars           1508 rows
  SeriesMetadata         2 rows

PriceBars (5 most recent MSFT daily rows)
  Symbol Timeframe Date        Open     High     Low      Close    Volume     Adjusted
  MSFT   Daily     2026-10-15  533.32   541.13   531.22   539.44   24151211   0
  MSFT   Daily     2026-10-14  545.27   545.75   532.54   533.69   22324295   0
  MSFT   Daily     2026-10-13  544.64   551.15   543.06   543.45   23851953   0
  MSFT   Daily     2026-10-12  537.53   542.74   535.70   541.15   18314943   0
  MSFT   Daily     2026-10-09  528.24   537.22   526.24   535.07   18500612   0

SeriesMetadata
   MSFT Daily Mixed 2026-10-16 09:48:08.8249678+00:00
   SPY Weekly Mixed 2026-10-16 09:48:11.4629681+00:00
PS C:\Users\tunga>
```

Figure 3: Tables of the SQLite cache and recent rows of `PriceBars`, read in a terminal (database of a console run of the data layer, same schema as the application).

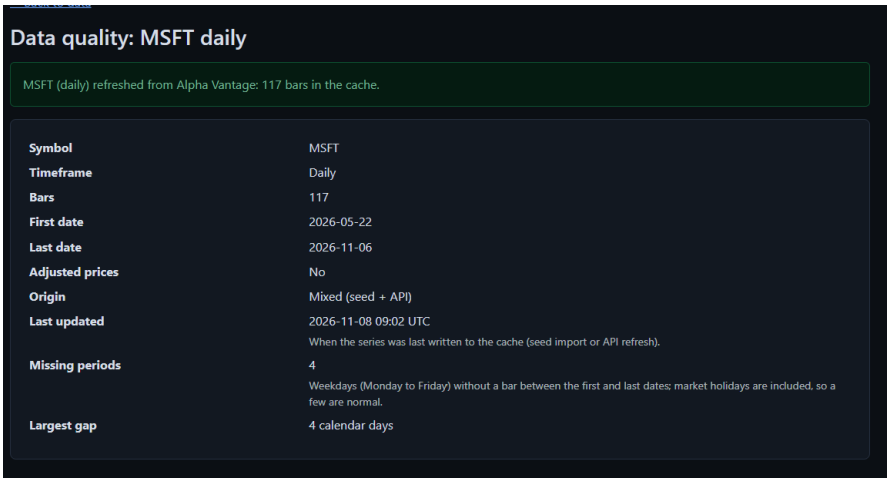


Figure 4: A real refresh of MSFT daily: the API bars are merged into the seed series (117 bars, origin `Mixed`).

3.3 Talking to Alpha Vantage safely

The project guide gave us a good starting point: `GetAsync`, then parsing the CSV by splitting on commas. While working on it, we found three traps.

Errors arrive with HTTP 200. With an invalid key or when the limit is reached, Alpha Vantage answers with status 200 and a small JSON document, even when CSV was requested. A simple test on the status code would save this JSON as if it were data. Our client detects a body starting with `{` and classifies it into `SymbolNotFound`, `RateLimitExceeded` or `InvalidResponse`.

Decimal separators. `double.Parse("123.45")` without a culture misreads the number on a computer configured in French, which is the default on our machines. The parser always uses the invariant culture, and one test runs under `fr-FR` to prove it.

The API key can leak. The key is part of the URL, and .NET logs request URLs by default. Worse, the rate limit message of Alpha Vantage repeats the key. So the key is only read from user-secrets or an environment variable, our error messages never copy the text of the API, and HTTP logging is removed:

```
services.AddHttpClient<AlphaVantageClient>(
        AlphaVantageClient.HttpClientName,
        client => client.Timeout = AlphaVantageClient.Timeout)
    .RemoveAllLoggers();
```

After a real refresh in the Docker container, we searched the whole log for the key, for `apikey=` and for the host name of the API, and found no occurrence of any of them.

3.4 Parsing, quota and concurrency

The parser accepts exactly two headers, raw and adjusted. A malformed line, or one that breaks the invariants of a bar (High $\geq \max(\text{Open}, \text{Close})$ and Low $\leq \min(\text{Open}, \text{Close})$), is dropped with a warning and never repaired.

The free plan allows 25 requests per day. Every HTTP request is counted per UTC day just before it is sent, so a failed request counts too. Simultaneous calls for the same series share one download, thanks to one lock per symbol and timeframe: ten simultaneous calls send one HTTP request.

3.5 Data quality

A backtest is only as good as its data. The quality report, computed on cached data only (rule D7), flags four kinds of anomalies: a **price jump** when the close moves by more than 25 % in one bar; a **zero volume**; a **stale price** at the fifth identical close in a row; and a **gap** when more than 5 calendar days (daily) or 10 days (weekly) separate two bars. It also counts the missing periods and the largest gap.

Figure 5 shows that it finds real events: the 11-day gap of September 2001, when US markets closed after the attacks of 11 September, and the violent moves of JP Morgan during the 2008 crisis. These are true market moves, so a flag is an invitation to look, not an error.

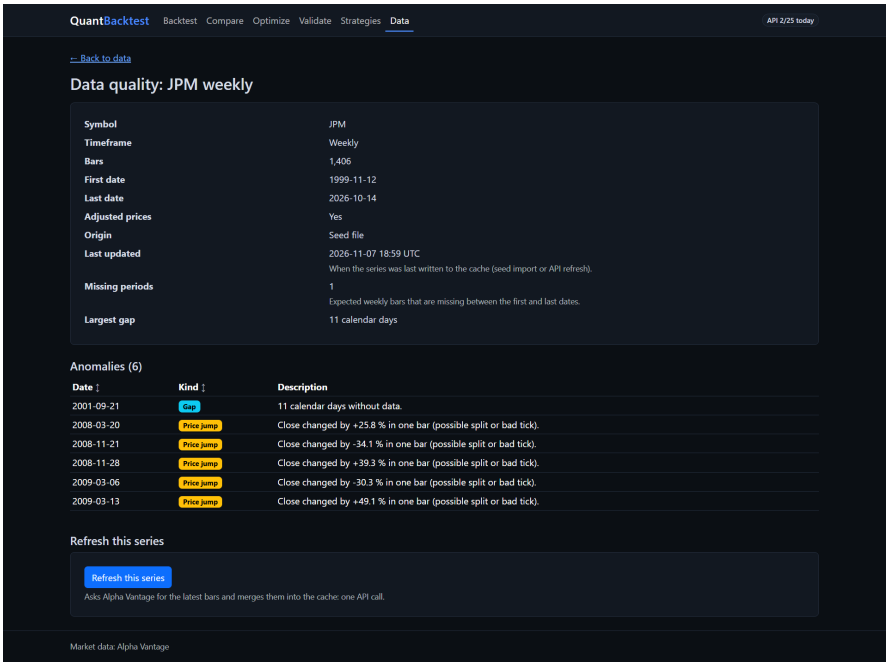


Figure 5: Quality report of JPM weekly: one gap and five price jumps.

4 Strategies

4.1 What a strategy is allowed to do

In our design, a strategy only gives an opinion. For each bar it returns `Buy`, `Sell` or `Hold`. It knows nothing about capital, fees or stops, which belong to the engine. The common rules are strict: a strategy is pure and deterministic (S1); it returns exactly one signal per bar, and `Hold` while its indicators warm up (S2); the signal of bar i may only depend on bars 0 to i (S3); and if the buy and sell conditions are both true on a bar, the signal is `Sell` (S4).

Classes are discovered by reflection, so adding a strategy does not require touching the web project. Each one comes with a card written for the user: the idea, the exact rules, and when it loses money (Figure 6).

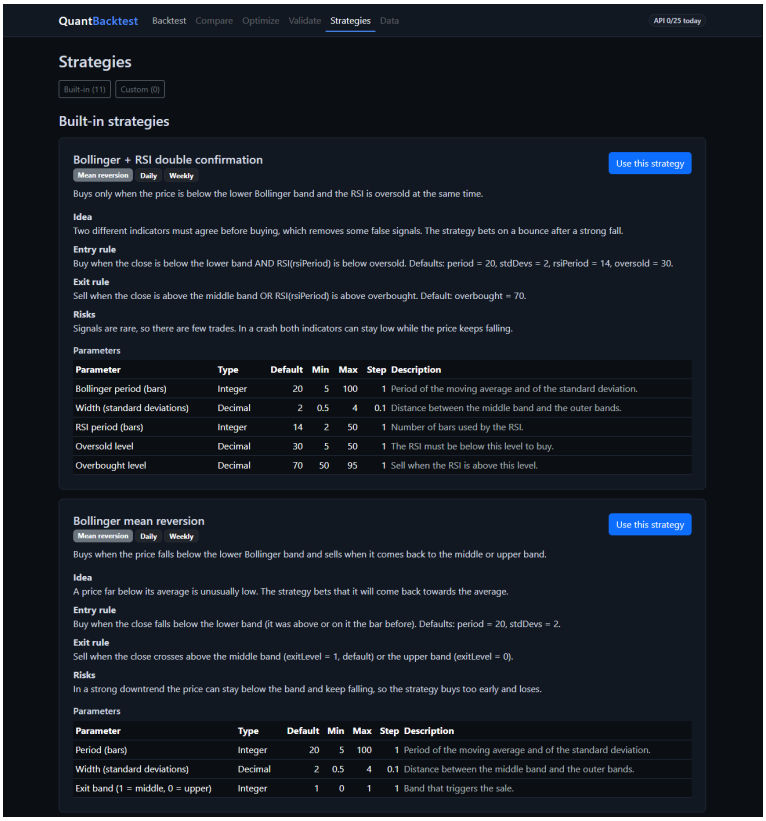


Figure 6: First two cards of the strategy catalog, generated from the strategies themselves.

4.2 Indicators

All strategies share a single implementation of the indicators, and the no-code engine uses it too, so two strategies cannot disagree on what “RSI 14” means.

Indicator	Definition
SMA(n)	Average of the last n closes
EMA(n)	$\alpha = 2/(n+1)$, started from the SMA of the first n values; then $\alpha x_i + (1-\alpha)\,\mathrm{EMA}_{i-1}$
Bollinger(n, k)	Middle band SMA(n); upper and lower bands at $\pm k$ population standard deviations over n
RSI(n)	Wilder's smoothing of average gains and losses, $100 - 100/(1 + \text{gains/losses})$, from 0 to 100
ROC(n)	Rate of change: $x_i/x_{i-n} - 1$
MACD(f, s, g)	Line: EMA(f) − EMA(s). Signal: EMA(g) of this line
Highest high / lowest low(n)	Over the n previous bars, the current bar being excluded

Table 4: Technical indicators (rules I1 to I9).

A *cross above* between two series a and b at bar i means $a_{i-1} \leq b_{i-1}$ and $a_i > b_i$, with all four values defined.

4.3 The implemented strategies

We chose strategies of different families, so that they do not all fail in the same conditions. For each one we give the rule with its default parameters. The "On SPY weekly" lines refer to Table 5.

Trend following and momentum. These strategies try to ride a movement that has already started.

Moving average crossover

Idea. When the recent average price rises above a longer average, the trend is turning up. The strategy rides the trend and leaves when it turns down.

Rules. Buy when SMA(20) crosses above SMA(50); sell when it crosses below.

Risks. Averages are late, so entries and exits come after the move has started. In a sideways market the averages cross often, which gives many small losses and fees.

On SPY weekly. the best risk-adjusted result of the table: only 9 trades, invested 70.9 % of the time.

MACD crossover

Idea. The MACD line is the gap between a fast and a slow exponential average. When it rises above its own average, the signal line, the trend is getting stronger.

Rules. Buy when the MACD(12, 26, 9) line crosses above the signal line; sell when it crosses below.

Risks. The MACD is late, like all moving averages, and in a sideways market the two lines cross often.

On SPY weekly. 59 trades for +94.0 %: more signals than the SMA crossover, for a much weaker result.

Momentum (rate of change)

Idea. A stock that has gone up recently tends to keep going up for a while.

Rules. Buy when ROC(14) crosses above the threshold (0 by default); sell when it crosses below.

Risks. The exit is late when a trend reverses suddenly, and in a choppy market the ROC oscillates around the threshold and gives false signals.

On SPY weekly. +258.5 % with 68 trades, the most active strategy of the table.

Donchian channel breakout

Idea. When the price breaks above the highest point of recent weeks, a new upward move often starts. The strategy enters on the breakout and leaves when the price breaks its recent low.

Rules. Buy when the close exceeds the highest high of the 20 previous bars; sell when it falls below the lowest low of the 10 previous bars.

Risks. Many breakouts fail at once, and the strategy buys after the price has already risen.

On SPY weekly. the smallest drawdown of the trend strategies (27.9 %) and a profit factor of 6.23.

Mean reversion. These strategies bet the opposite: a price that moved too far tends to come back.

Bollinger mean reversion

Idea. A price far below its average is unusually low and should come back towards the average.

Rules. Buy when the close falls below the lower band (20 bars, 2 standard deviations) after being above or on it; sell when the close crosses above the middle band.

Risks. In a strong downtrend the price can stay below the band and keep falling, so the strategy buys too early.

On SPY weekly. invested only 14.2 % of the time, with a win rate of 80 %.

RSI mean reversion

Idea. A very low RSI means the stock fell a lot in a short time, so a bounce is likely.

Rules. Buy when RSI(14) is below 30; sell when it is above 70.

Risks. In a long downtrend the RSI can stay low for a long time while the price keeps falling.

On SPY weekly. only 4 trades in 27 years, far too few to conclude anything.

Bollinger + RSI double confirmation

Idea. Two different indicators must agree before buying, which removes some false signals.

Rules. Buy when the close is below the lower band and RSI(14) is below 30; sell when the close is above the middle band or the RSI is above 70.

Risks. Signals are rare, and in a crash both indicators can stay low while the price keeps falling.

On SPY weekly. 6 trades, invested 5.0 % of the time.

Trend-filtered RSI-2 (Connors)

Idea. When the long-term trend is up, short dips are often bought back quickly. A long average checks the trend and a 2-bar RSI finds the dips.

Rules. Buy when the close is above SMA(200) and RSI(2) is below 10; sell when the close is above SMA(5).

Risks. If the trend breaks, a small dip can become a big fall. Trades are short and frequent, so fees matter a lot.

On SPY weekly. the smallest drawdown of the table (23.5 %), with a win rate of 76.2 %.

Consecutive down days

Idea. After several falls in a row, sellers are often tired and the price bounces back.

Rules. Buy when the close has fallen 3 times in a row (and the total fall reaches a minimum, 0 by default); sell at the first close higher than the previous one.

Risks. In a crash the price can go down many more days in a row, and short trades make fees important.

On SPY weekly. the weakest result of the table (+30.0 %).

Calendar effect and baseline.

Turn of the month

Idea. Stocks often rise around the turn of the month, when salaries and funds are invested.

Rules. Buy when exactly 1 business day remains until the end of the month; sell on the 3rd bar of the new month. Daily data only. Bank holidays are not known to the application, so this is an approximation.

Risks. The effect is small and can disappear for many months. Two trades per month make fees important.

Buy and hold

Idea. Markets tend to rise over the long term. Buying once and doing nothing avoids fees and timing mistakes, so every active strategy should try to do better.

Rules. Buy on the first bar and never sell; the position is closed at the end of the period.

Risks. It takes every market crash in full.

On SPY weekly. +781.5 %, but with a drawdown of 54.6 %.

4.4 Results on SPY

Table 5 shows the strategies on SPY weekly (1,406 bars, November 1999 to October 2026), with default parameters, a capital of 10,000 and a fee of 0.1 % per fill.

Strategy	Return	Ann.	Vol.	Sharpe	Max DD	Trades	Exposure
Moving average crossover	+759.4 %	8.32 %	12.4 %	0.70	31.8 %	9	70.9 %
Donchian breakout	+300.9 %	5.29 %	9.8 %	0.57	27.9 %	22	61.0 %
Buy and hold	+781.5 %	8.42 %	17.6 %	0.55	54.6 %	1	99.9 %
Momentum (ROC)	+258.5 %	4.86 %	11.1 %	0.48	38.5 %	68	68.7 %
Trend-filtered RSI-2	+111.7 %	2.83 %	7.7 %	0.40	23.5 %	42	9.2 %
RSI reversion	+156.9 %	3.57 %	11.1 %	0.37	43.4 %	4	20.6 %
MACD crossover	+94.0 %	2.49 %	10.2 %	0.29	40.9 %	59	53.3 %
Bollinger + RSI	+56.6 %	1.68 %	8.6 %	0.24	37.4 %	6	5.0 %
Bollinger reversion	+67.7 %	1.94 %	11.2 %	0.23	41.4 %	20	14.2 %
Consecutive down days	+30.0 %	0.98 %	7.2 %	0.17	39.8 %	57	7.3 %

Table 5: Strategies on SPY weekly, default parameters, sorted by Sharpe ratio. Ann.: annualized return. Vol.: annualized volatility. Max DD: maximum drawdown. Turn of the month accepts daily data only and is absent.

Three observations stand out. First, only the moving average crossover comes close to buy & hold in return, with a much smaller drawdown, because it stays out of the market during long declines. Second, the mean reversion strategies are rarely invested on weekly bars (between 5 % and 21 % of the time), which explains their low returns: they cannot earn anything while they stay out. Third, several results rest on very few trades, so this table alone does not prove that one strategy is better. Section 6 deals with this problem.

One detail deserves an explanation: the buy & hold *strategy* returns +781.5 % while the buy & hold *benchmark* of the same backtest returns +800.7 %. It is not a bug. The benchmark buys at the Open of the first bar (rule X14), whereas a signal at the close of the first bar is only executed at the Open of the second bar (rule X4). The strategy also pays a fee to exit at the end of the period, while the benchmark is never sold.

4.5 Custom strategies

In C#. A fully commented template is hidden from the catalog by a `[HiddenStrategy]` attribute. A developer copies it, removes the attribute, changes the identifier, and writes the documentation, the parameters and the rules. The strategy then appears in every form of the website.

Without code. Writing C# is not realistic for every user, so we added a rule builder (Figure 7). A strategy has an entry and an exit rule, each with 1 to 5 conditions combined by “all” or “any”. A condition compares two operands with `>`, `<`, “crosses above” or “crosses below”. An operand can be a price, the volume, a constant, or an indicator with its period.

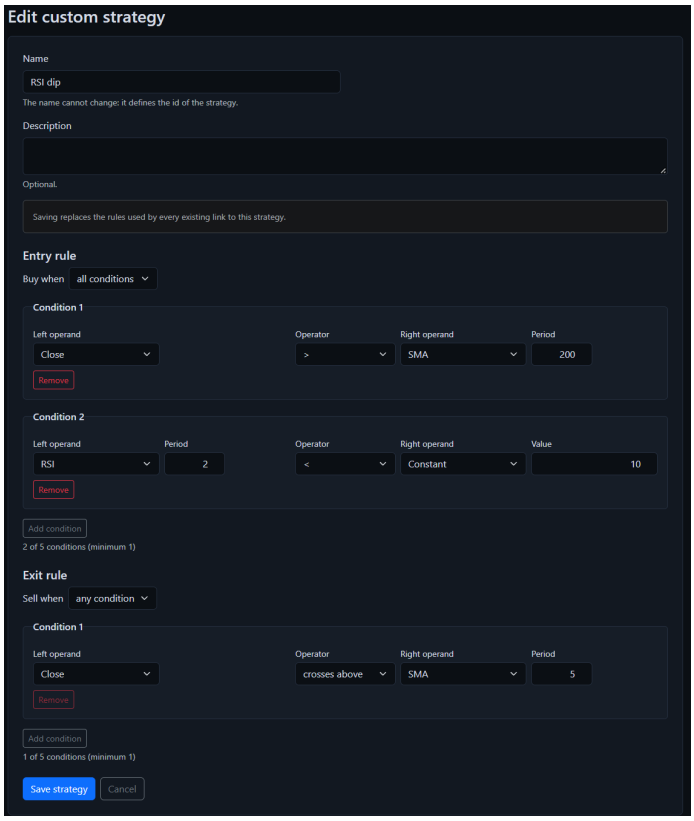


Figure 7: The no-code rule builder, editing the strategy "RSI dip".

The factory validates a definition and returns *all* its problems at once, each with its position, for example "Entry condition 2: the right operand needs a period between 1 and 500.". A valid definition becomes a normal `IStrategy`, with documentation generated from its rules, such as "Buy when ALL of: Close > SMA(200); RSI(2) < 10". It is stored as JSON in SQLite. A condition is false as long as one of its values is undefined, so warm-up periods need no special handling.

5 Backtesting engine

5.1 Execution model

The most common mistake in a backtest is to use information that was not yet available. Our model is deliberately simple. A strategy gives its signal at the close of a bar, and the engine executes it at the **Open of the next bar** (X4). A signal therefore cannot use a price that did not exist yet. Positions are long only, without leverage, and use all the cash (X5).

With a price p, a slippage s and a fee rate f, rule X6 gives:

$$\text{fill}_{\text{buy}} = p\,(1+s), \qquad \text{fill}_{\text{sell}} = p\,(1-s),$$

$$q = \frac{\text{cash}}{\text{fill}_{\text{buy}}\,(1+f)}, \qquad \text{proceeds} = q\,\text{fill}_{\text{sell}}\,(1-f).$$

For every bar i of the period, the engine applies these steps in order (X7): execute at the Open the signal of bar $i-1$; if a position is open, check the protective exits; on the last bar, sell any open position at the Close; and record the equity, cash $+\, q \times$ Close. Bars before the requested period are given to the strategy to warm up its indicators, but their signals are ignored (X2).

A worked example. With a capital of 10,000, a fee of 0.1 %, a buy at 102 and a sale at 106, the quantity is $10{,}000/(102 \times 1.001) = 97.9413$. The proceeds are $97.9413 \times 106 \times 0.999 = 10{,}371.39$, so the net profit is 371.39, after 20.37 of fees. Figure 8 shows this scenario, computed independently from the engine.

Scenario S1 - signals B H S H H H - hand check (capital 10 000, fee 0.1 %, no slippage)

To compare with the engine tests: quantity 97.94, net profit 371.39, fees 20.37, equity 10087.95 / 10283.83 / 10371.39

Capital	10000,		input
Fee rate	0,001		input (0.1 %)
Entry price (Open of bar 1)	102,		input: Buy signal on bar 0, executed at the next Open
Exit price (Open of bar 3)	106,		input: Sell signal on bar 2, executed at the next Open
Quantity	97,9413		capital / (entry x (1 + fee)) [rule X6]
Cost of the purchase	10000,0000		q x entry x (1 + fee)
Proceeds of the sale	10371,3933		q x exit x (1 - fee) [rule X6]
Fees (entry + exit)	20,3718		q x entry x fee + q x exit x fee
Net profit	371,3933		proceeds - cost [rule X13]
Return of the trade	0,0371		net profit / cost

Bar	Close	Equity	How it is computed
Bar 0	101	10000,0000	first point = capital [rule X11]
Bar 1	103	10087,9513	bought at the Open of bar 1, valued at the Close: q x Close
Bar 2	105	10283,8338	q x Close
Bar 3	104	10371,3933	sold at the Open of bar 3: everything is cash
Bar 4	108	10371,3933	cash
Bar 5	110	10371,3933	cash

Figure 8: Six-bar scenario recomputed by hand: quantity, fees, net profit and equity per bar.

5.2 Advanced orders

Let E be the entry fill and *peak* the highest High since the entry, before the current bar. The stop level is $S = \max\left(E(1-\mathrm{SL}),\ \mathrm{peak}(1-\mathrm{TS})\right)$ and the target is $T = E(1+\mathrm{TP})$. Rule X8 then applies:

- **Gap** (not on the entry bar): if the bar opens at or below S, or at or above T, the exit is at the Open, because the level could not be reached.
- **During the bar:** if Low $\leq S$ the exit is at S; otherwise, if High $\geq T$, the exit is at T.

When both levels are touched in one bar, the order is unknown with daily or weekly data. We chose the prudent assumption: the stop always comes first. The exit price then goes through the slippage and the fee like any other sale (Figure 9).

Trades

Entry date	Entry price	Exit date	Exit price	Quantity	Fees	Net profit	Return	Exit reason	Holding bars
2001-08-31	$0.28	2001-09-07	$0.26	35,916.1669	$19.32	-$679.95	-6.80%	Trailing stop	1
2002-03-01	$0.34	2002-03-01	$0.31	27,246.1417	$17.88	-$762.74	-8.18%	Stop-loss	0
2003-06-13	$0.25	2003-07-18	$0.32	33,746.6475	$19.23	$2,117.96	24.75%	Take-profit	5
2006-11-17	$2.49	2006-12-22	$2.51	4,284.7964	$21.41	$58.45	0.55%	Trailing stop	5
2008-07-11	$5.17	2008-07-25	$4.87	2,072.4141	$20.81	-$651.86	-6.07%	Trailing stop	2
2009-07-02	$4.29	2009-09-18	$5.36	2,347.3624	$22.66	$2,495.29	24.75%	Take-profit	11
2013-11-08	$16.12	2014-01-31	$16.01	779.4505	$25.05	-$108.87	-0.87%	Trailing stop	12
2016-10-14	$26.30	2016-11-11	$24.43	473.5913	$24.02	-$911.92	-7.31%	Trailing stop	4
2019-07-12	$48.00	2019-08-09	$47.50	240.5368	$22.97	-$141.47	-1.22%	Trailing stop	4
2023-05-05	$166.58	2023-08-11	$175.80	68.4575	$23.44	$608.11	5.33%	Trailing stop	14
2024-07-05	$210.10	2024-08-09	$197.22	57.1683	$23.29	-$759.50	-6.32%	Trailing stop	5
2025-10-24	$254.94	2026-01-09	$259.05	44.1373	$22.69	$158.65	1.41%	Trailing stop	11

Figure 9: Trade log of a moving average crossover on AAPL weekly with an 8 % stop-loss, a 25 % take-profit and a 10 % trailing stop.

5.3 Metrics

With P the number of periods per year (252 or 52), E_t the equity, I the initial capital, $r_t = E_t/E_{t-1} - 1$ and r_f the risk-free rate per period:

Metric	Formula
Cumulative return	$E_{\text{final}}/I - 1$
Annualized return	$(E_{\text{final}}/I)^{1/\text{years}} - 1$, with years = days/365.25
Volatility	sample standard deviation of r_t, times $\sqrt{P}$
Sharpe ratio	$(\overline{r} - r_f) \,/\, \sigma(r) \times \sqrt{P}$
Sortino ratio	$(\overline{r} - r_f) \,/\, DD \times \sqrt{P}$, with $DD = \sqrt{\frac{1}{n}\sum \min(0, r_t - r_f)^2}$
Maximum drawdown	largest (peak $- E_t$)/peak, the peak starting at I
Calmar ratio	annualized return / maximum drawdown
Win rate	share of trades with a positive net profit
Profit factor	sum of gains / absolute sum of losses
Exposure	share of the points where a position is held

Table 6: Main performance metrics (rules M1 to M12). The benchmark uses the same formulas.

A ratio that cannot be defined, such as the Sharpe ratio of a flat curve, is `null` and displayed as "n/a". Replacing it with 0 would present an undefined value as a real result.

5.4 Optimisation heatmap

The heatmap tests every pair of values of two parameters, up to 625 cells. Prices are loaded once, then each cell goes through the same validation and engine as a normal backtest. An invalid pair (for example `fast` $\geq$ `slow`) gives an empty cell with its message and does not stop the grid.

An optimiser always finds a best cell, even on noise. So if the best value is positive and the average of its neighbours is below half of it (rule O4), the page shows an "isolated peak" warning. In Figure 10, the best profit factor is 110.28, a suspicious number which the warning rightly flags.

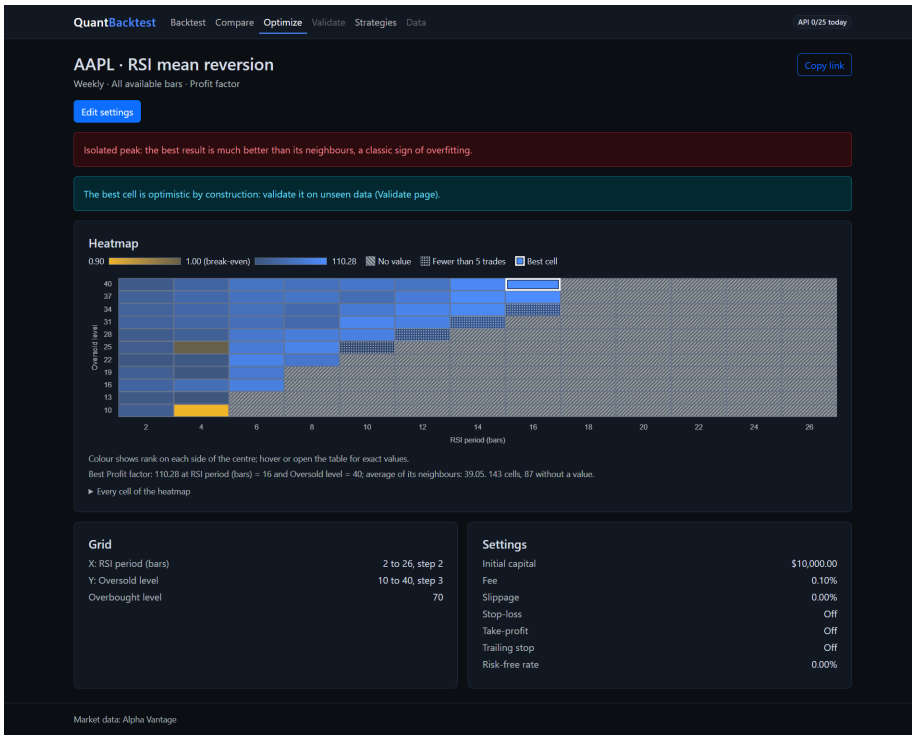


Figure 10: Heatmap of RSI mean reversion on AAPL weekly, with the isolated peak warning.

6 Robustness analysis

A good backtest proves little on its own: the result can come from luck or from fitting the past. We added two tools to measure this.

6.1 Train/test validation

The Validate page splits the selected period in two (rules T1 to T3). The first $\lfloor N \times \text{train} \rfloor$ bars, 70 % by default, form the training period. The page runs the heatmap on them only, takes the best cell, and tests those parameters once on the remaining bars. The test period may use earlier bars to warm up its indicators, which is not an information leak since every signal still depends only on past bars.

On AAPL weekly since 2010 (Figure 11), the 876 bars split into 613 training bars and 263 test bars. The best parameters are fast 26 and slow 35. The Sharpe ratio falls from 1.02 in training to 0.75 in test. This drop is expected, since the best cell was selected for being the best on those data. With 5 and 6 trades, neither number is statistically solid.

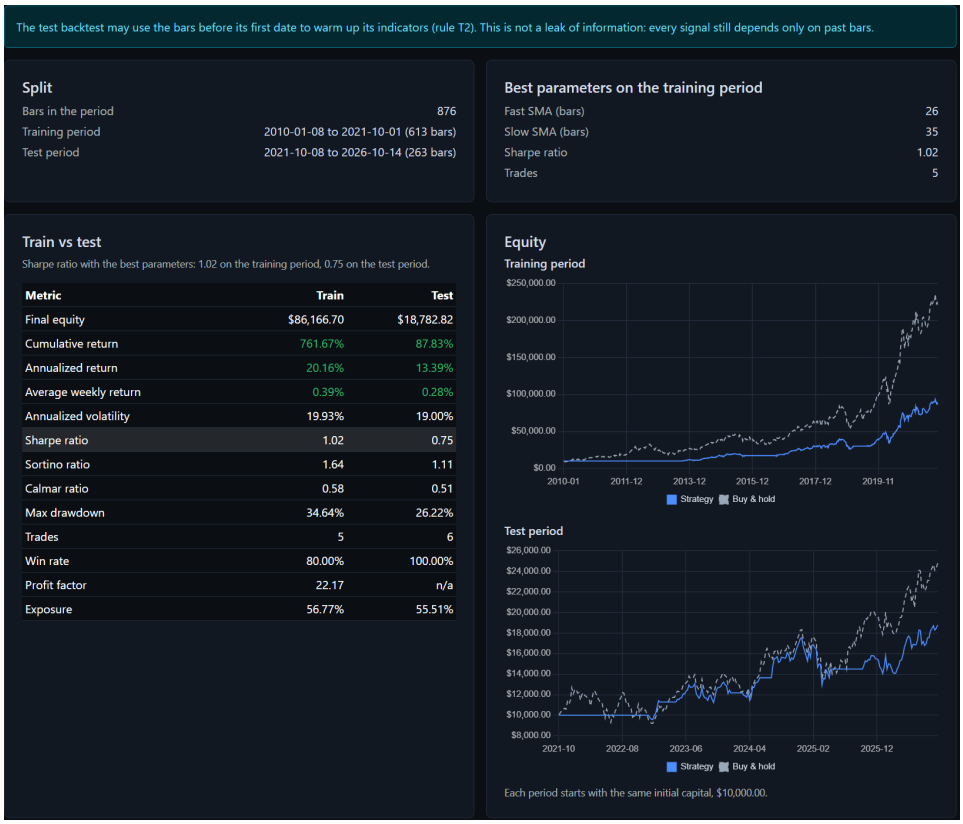


Figure 11: Train/test validation of the moving average crossover on AAPL weekly.

6.2 Monte Carlo simulation

The Monte Carlo analysis asks how much of a result depends on the particular sequence of trades. We take the returns of the k trades of a backtest (at least 5) and draw k of them at random with replacement. The trajectory starts at $V_0 = 1$ and compounds: $V_j = V_{j-1}(1 + r)$. This gives a final return and a maximum drawdown. Repeating it between 100 and 10,000 times gives two distributions, summarised by percentiles and histograms (MC1 to MC6). One generator seeded by the user is used, so the same seed always gives the same result.

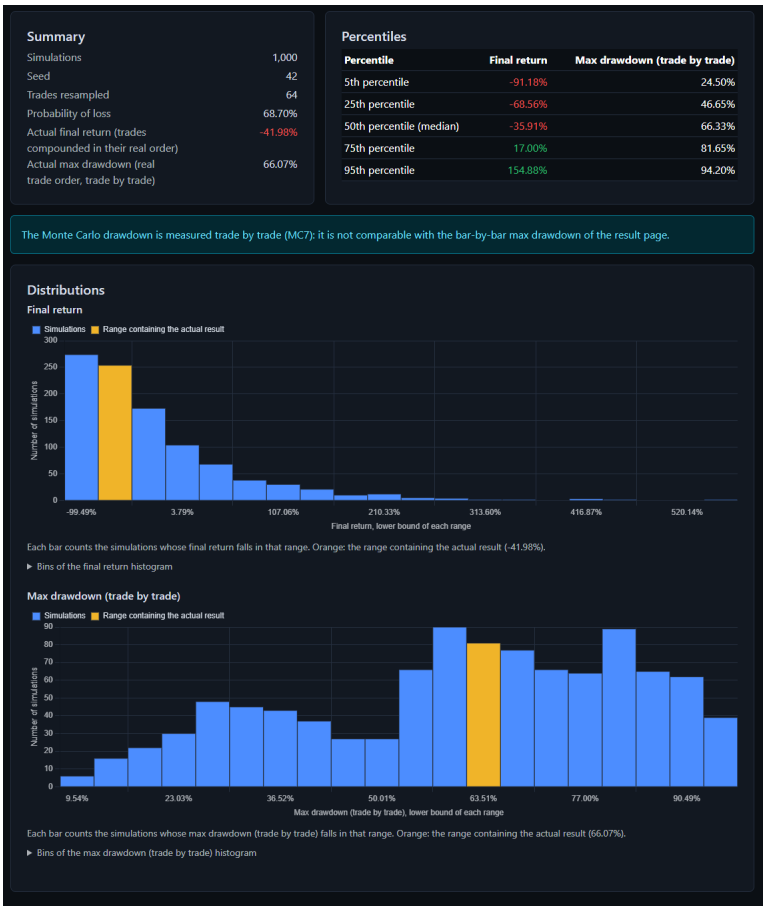


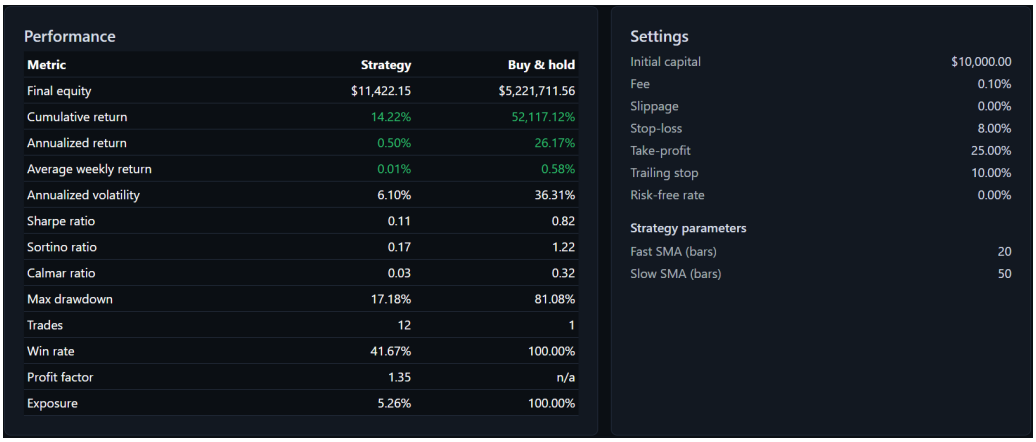
Figure 12: Monte Carlo of *Consecutive down days* on AAPL weekly (64 trades, 1,000 simulations).

In Figure 12, the strategy has a 68.7 % probability of loss, and its actual result (−41.98 %) is near the median of the distribution: it is simply a losing strategy, and bad luck does not explain it. Conversely, the moving average crossover on MSFT weekly (Figure 16) has an 8.5 % probability of loss, but its percentiles range from −17.4 % (P5) to +29,111 % (P95). With 15 trades, one of which earned +521 %, the result depends heavily on how many times this single trade is drawn.

This drawdown is measured trade by trade (rule MC7). It is not comparable with the bar-by-bar maximum drawdown of the results page, and both the interface and the PDF say so.

7 Web application

The application uses ASP.NET Core MVC. The navigation has six entries: Backtest, Compare, Optimize, Validate, Strategies and Data. A session typically starts on the Backtest page. The results page shows the 13 metrics against buy & hold, four charts drawn with Chart.js and a sortable trade log (Figures 13 and 14). From there, the user can download the PDF or the trades, run a Monte Carlo, optimise the strategy, or copy a permanent link.



Performance

Metric	Strategy	Buy & hold
Final equity	$11,422.15	$5,221,711.56
Cumulative return	14.22%	52,117.12%
Annualized return	0.50%	26.17%
Average weekly return	0.01%	0.58%
Annualized volatility	6.10%	36.31%
Sharpe ratio	0.11	0.82
Sortino ratio	0.17	1.22
Calmar ratio	0.03	0.32
Max drawdown	17.18%	81.08%
Trades	12	1
Win rate	41.67%	100.00%
Profit factor	1.35	n/a
Exposure	5.26%	100.00%

Settings

Initial capital	$10,000.00
Fee	0.10%
Slippage	0.00%
Stop-loss	8.00%
Take-profit	25.00%
Trailing stop	10.00%
Risk-free rate	0.00%

Strategy parameters

Fast SMA (bars)	20
Slow SMA (bars)	50

Figure 13: Results of a real backtest: AAPL weekly, moving average crossover with stops.

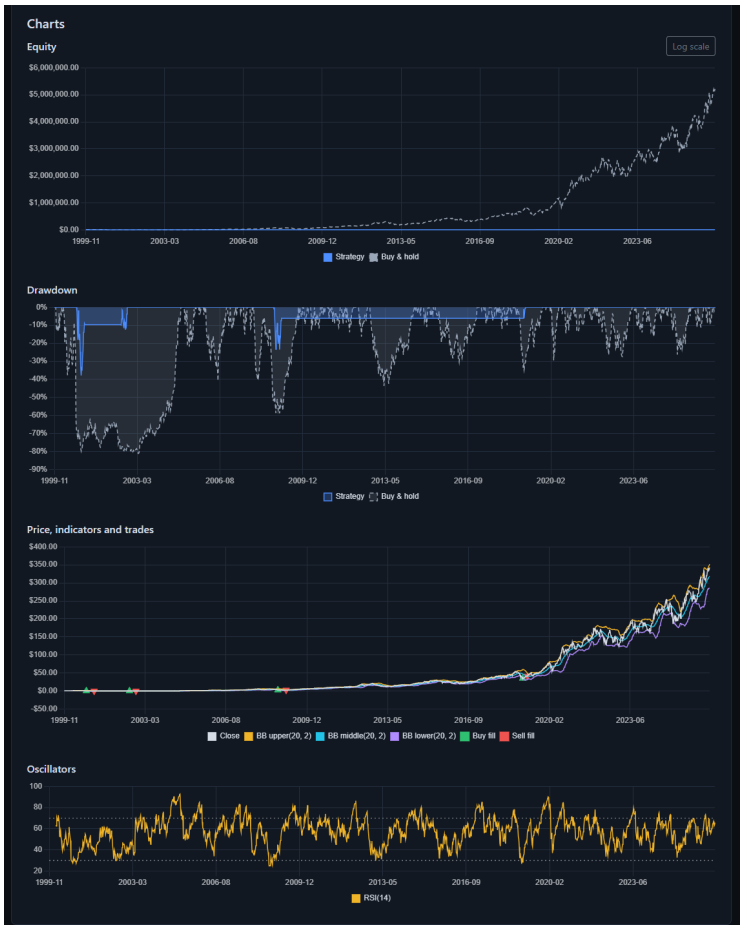


Figure 14: Charts of a result: equity, drawdowns, price with indicators and trade markers, and an oscillator.

Our main design choices were the following:

- **No financial logic in the web project.** Controllers call the services and view models format the values. The only computations concern presentation and the train/test split.
- **Nothing hard-coded about strategies.** Forms, lists and cards are generated from the catalog, so a new strategy appears without any change in the web project.

- **Only the contract exceptions are caught.** Validation and market data errors become an alert above the form, with the input kept. Anything else is a bug and reaches a generic error page.
- **Forms that work without JavaScript.** The server renders and validates them; JavaScript modules only improve them.
- **Local libraries.** Bootstrap, Chart.js and its matrix plugin are served by the application, with no CDN.

Comparator. The Compare page runs 2 to 5 strategies with default parameters on the same data, and draws their equity curves with buy & hold (Figure 15). If one strategy fails, its message is displayed without blocking the others.



Strategy	Cumulative return	Annualized return	Annualized volatility	Sharpe ratio	Sortino ratio	Calmar ratio	Max drawdown	Trades	Profit factor	Exposure
Momentum (rate of change)	149.16%	8.06%	21.07%	0.47	0.67	0.21	38.24%	31	2.08	66.72%
Donchian channel breakout	273.89%	11.84%	17.96%	0.71	1.12	0.53	22.49%	12	6.06	56.49%
MACD crossover	288.68%	12.21%	17.93%	0.73	1.15	0.47	26.09%	26	3.35	52.27%
Buy & hold	1,261.88%	24.82%	27.41%	0.96	1.47	0.72	34.64%	1	n/a	100.00%

Figure 15: Comparator on AAPL weekly since 2015. On this symbol, buy & hold returned +1,261.88 %, far more than any strategy.

Permanent links. Every result has a `GET` URL containing all its settings. One class, `BacktestQuery`, reads and writes these URLs, and redirects to a single canonical form. Since the cache is not refreshed automatically, a link gives the same result later, as long as nobody refreshes that series.

8 Reports and deployment

8.1 PDF report

The report is generated with QuestPDF, and its charts are drawn with ScottPlot. Every number is printed as it comes in `BacktestResult` and `MonteCarloResult`: the report recomputes nothing, so it cannot disagree with the screen. It contains a summary page, the drawdown and price charts, the complete trade log, and an optional Monte Carlo page (Figure 16). Numbers are always formatted in en-US, and the date in the footer comes from the injected clock, which makes tests reproducible.

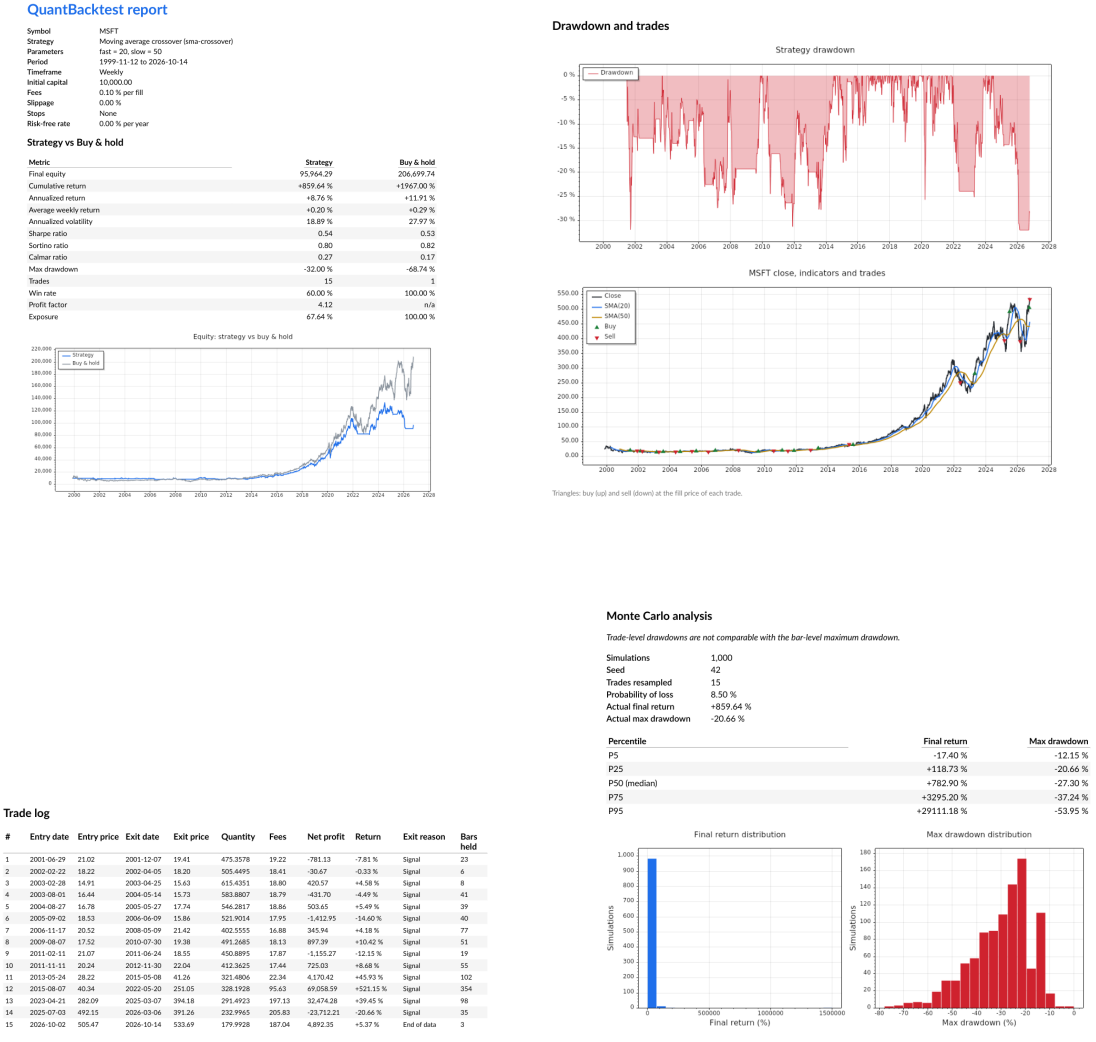


Figure 16: PDF report for MSFT weekly (SMA 20/50): summary, charts, trade log and Monte Carlo page.

We checked one real report by hand. For MSFT weekly, the 15 net profits add up to 85,964.29, so the final equity is $10{,}000 + 85{,}964.29 = 95{,}964.29$, which is the printed value. The cumulative return, annualized return, Calmar ratio, win rate and profit factor also match when recomputed from the printed values. Trades can also be downloaded as CSV.

8.2 Docker

`docker compose up -build` starts the application on port 8080. The `Dockerfile` has two stages. The .NET 10 SDK stage builds with warnings treated as errors and runs every test, so an image can only exist for a solution whose tests pass. The ASP.NET runtime stage runs the application as a non-root user, with the fonts needed by the charts. The database is in a named volume, the seed files are copied into the image, and the API key comes from a local `.env` file which is never committed.

We tested this with Docker Desktop: the image built with its tests, the application answered on port 8080, the database was created in the volume, a PDF with its Monte Carlo page was generated, and a real API refresh succeeded.

9 Quality and teamwork

9.1 Tests

The solution has 1,494 automated tests written with xUnit and FluentAssertions (Table 7). They use no network and no real clock, and any randomness has a fixed seed. Each test quotes the identifier of the rule it checks. In the engine, expected values are recomputed from the formula in the test, never copied from the output, and property tests run random signals to check invariants: the equity is never negative, the final equity equals the capital plus the profits, trades never overlap, and truncating the future does not change the past.

Project	Tests	Source lines	Test lines
Web	923	7,127	9,384
Engine	184	1,023	2,427
Strategies	168	2,029	1,662
Data	124	1,201	1,563
Reports	55	771	577
Analytics	14	212	217
Fakes	14	981	144
Core	12	713	70
Total	**1,494**	**14,057**	**16,044**

Table 7: Automated tests and lines of C# per project.

Razor views, JavaScript and CSS were checked by hand in a browser, and the text of a PDF cannot be read back by the tests, so it is tested before layout.

9.2 Workflow and continuous integration

The main branch always had to compile and pass its tests. All changes arrived by pull request, from a branch named after its part, and 42 were merged between 13 October and 8 November 2026. GitHub Actions runs two checks on each of them (Figure 17): one compiles with warnings as errors and runs all the tests, the other checks the format of the title and verifies that the changed files belong to the part of the branch. Branch protection requires these checks and forbids force-pushes.

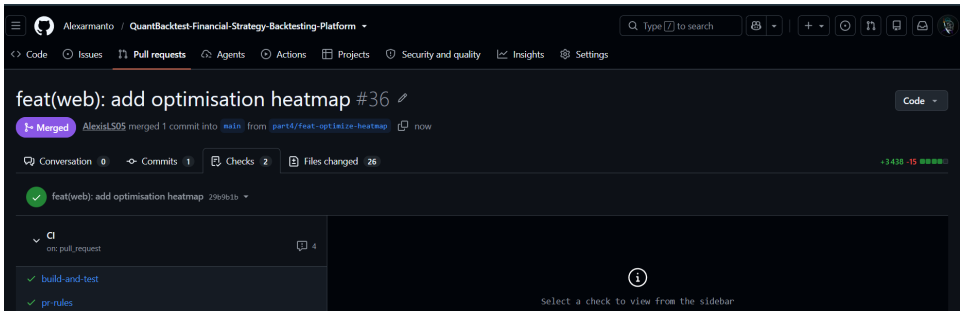


Figure 17: Checks of a pull request.

At first, each pull request needed an approval from another member. On 15 October we removed this requirement because it slowed the team down too much, while keeping it for contract changes.

With hindsight this was a compromise: the CI guarantees that the code compiles and passes its tests, but it does not replace a human reading the code.

9.3 What we learned

Our most useful decision was the first one: spending time on a precise contract before writing code, which allowed four people to work in parallel without ever changing it. We also learned that a backtest can be wrong in silent ways, such as a look-ahead, a misread decimal or an error message saved as data. For that reason, the tools that show the limits of a result (train/test, Monte Carlo, the overfitting warning, the data quality report) ended up being as important to us as the strategies themselves.

10 Limitations and future work

Data. The free plan gives only 100 daily bars, and they are not adjusted, so a split in this window would appear as a price jump. Long tests have to be run in weekly. The seed files are snapshots of October 2026, their last weekly bar is a week that was still in progress, and they are included for this academic project only. A symbol refreshed before its seed file is imported bypasses the seed for good. A paid source would give long adjusted daily histories.

Strategies. Strategies are long only. *Turn of the month* ignores market holidays, and the no-code builder cannot express every rule.

Engine. There is no short selling and no leverage. Orders are filled at the next Open without any volume limit, which is optimistic for large orders. Inside a bar, the order of the High and the Low is unknown, so the “stop first” rule remains an assumption. Cash earns no interest and taxes are ignored.

Robustness. Train/test uses a single split, and walk-forward validation, with several successive splits, would be a natural improvement. Monte Carlo resamples the trades of a single backtest and assumes they are independent, which ignores market regimes. Its equal-width histograms can be dominated by one extreme value.

Web application. Only the equity chart has a logarithmic scale, so old prices are squashed near zero on long histories. The comparator uses default parameters. The best cell of a heatmap may rest on very few trades. Views and JavaScript have no automated tests.

Method. Our demonstration symbols are large, successful US companies and an index, chosen with hindsight. This is a survivorship and selection bias: on such symbols buy & hold is very hard to beat, which says more about the symbols than about the strategies. And in all cases, past results do not predict future results.